Coil — Reference

Coil (Curly-Over-IL) is a minimal curly-brace language that compiles **as close to 1-to-1 with .NET IL (CIL) as a readable language can get**. Every operator is one IL opcode, every local is a real IL local, every function is a `public static` method on a public type `CoilProgram` — so a Coil assembly is referenceable from **C# and VB.NET** like any other .NET library.

It is built with the self-hosted toolchain: the **front end** is written in `lex` + `yacc` (compiled by `cc` to .NET IL) and lowers Coil to a flat **stack-IL IR**; the **assembler** (`coilasm`, C# + `System.Reflection.Emit`) turns that IR into a real assembly. Because the IR *is* stack CIL, the translation is transparent — this reference shows you the IL as it goes.

```
coilfe prog.coil -o prog.exe        # compile to a runnable assembly (dotnet prog.exe)
coilfe lib.coil  -o lib.dll --dll   # compile to a library for C#/VB.NET
```

Quick Reference

```
func name(int a, double b) → int { ... }   // function; → type optional (default void)
func main() → void { ... }                  // entry point for an exe

int x = 10;          double r = 3.5;          // typed local
bool ok = true;      string s = "hi";
var  n = x * 2;      // type inferred from the initializer

x = x + 1;                                    // assignment
if (cond) { ... } else { ... }                // else optional; else-if chains
while (cond) { ... }                          // the only loop
return expr;   return;                        // return a value / from void
print(expr);   println(expr);                 // output (no newline / newline)

// operators (high to low): unary - !  *  /  %   +  -   < ≤ > ≥ == ≠   &&   ||
// comments: // to end of line
```

Data types

Coil has exactly five types; each is a CLR primitive, so they cross the C#/VB.NET boundary with their natural signatures.

Coil	.NET type	Literals	IL load
`int`	`System.Int32`	`0`, `42`, `-7`	`ldc.i4`
`double`	`System.Double`	`3.5`, `1e3`, `2.0`	`ldc.r8`

Coil	.NET type	Literals	IL load
`bool`	`System.Boolean`	`true`, `false`	`ldc.i4`
`string`	`System.String`	`"hi"`, `"tab\there"`	`ldstr`
`void`	`System.Void`	— (function results only)	—

`int` promotes to `double` automatically in mixed arithmetic, on assignment, and at call sites (a `conv.r8` is inserted). There is no implicit `double` → `int`. String escapes: `\n \t \r \\ \"`.

Statements / Commands

- **Declaration** — `int x = expr;` (typed) or `var x = expr;` (inferred). Becomes an IL local plus the initializer and a `stloc`.
- **Assignment** — `x = expr;` → the expression then `stloc` / `starg`.
- `if (cond) { ... }` with optional `else { ... }` or `else if`. Lowers to `brfalse` / `br` over labels.
- `while (cond) { ... }` — the only loop; lowers to a back-edge `br` with a guard `brfalse`.
- `return expr;` / `return;` → `ret` (with the value converted to the return type).
- `print(expr);` / `println(expr);` — see Input / Output.
- **Expression statement** — e.g. a call; its value (if any) is `pop`ped.

Functions

```
func gcd(int a, int b) -> int {
    while (b != 0) { int t = b; b = a % b; a = t; }
    return a;
}
```

A function is `func name(params) -> rettype { body }`; omit `-> rettype` for `void`. Each becomes `public static rettype name( ... )` on type `CoilProgram`. Functions may be mutually recursive and used before they are defined (the front end registers all signatures in a first pass). `func main()` (parameterless) is the entry point of an executable.

The 1-to-1 mapping. `func add(int a, int b) -> int { return a + b; }` compiles to:

```
.method public static int32 add(int32 a, int32 b)
    ldarg.0      // a
    ldarg.1      // b
    add
    ret
```

Input / Output

`print(x)` and `println(x)` call `System.Console.Write` / `WriteLine` with the overload matching `x`'s type (`int`, `double`, `bool`, `string`). Output therefore uses .NET formatting: a `bool` prints `True` / `False`, a `double` prints its round-trip form. There is no input statement in the minimal subset (see *Subset boundaries*).

Graphics

None. Coil has no graphics surface; the "drawing" activity below is **text art** (printed with `print` / `println`).

Notes

- **Two-stage compiler.** `coilfe` (lex + yacc, itself compiled to IL by `cc`) parses and type-checks Coil and emits a textual stack-IL IR (`prog.ir`); `coilasm` (C#/`Reflection.Emit`) assembles the IR into a real PE. The IR is essentially CIL, which is what makes Coil "1-to-1 with IL."
- **No optimization.** Code generation is a straight syntax-directed walk: `a + b` is always `<a> <b> add`. What you write is what the IL does.
- **Operators.** Arithmetic `+ - * / %`; comparisons `< ≤ > ≥ == ≠` yield `bool`; logical `&& ||` are short-circuit; unary `-` (negate) and `!` (logical not).
- **Strings.** `+` concatenates when either side is a string; non-string operands are boxed and joined with `String.Concat` — so `"n = " + 42` works (`42` is boxed). `==` / `≠` on strings compares **contents** (`String.Equals`), not references.
- **Interop both ways.** C#/VB.NET can call Coil functions (they are ordinary static methods); Coil can call `Console` via `print`. Calling arbitrary .NET methods from Coil is intentionally out of scope (see boundaries).

Subset boundaries

Coil is deliberately minimal — a clean window onto IL, not a general language. It does **not** have: arrays or collections; `for` / `do` loops (only `while`); input; user-defined types/structs/classes; global variables (state lives in locals/params); the ability to call arbitrary .NET methods (only `print` / `println`); numeric types beyond `int` / `double` (no `long`, `char`, unsigned); or exceptions. These are natural extensions — each maps to a small set of IL opcodes — but are left out to keep the language one screen of grammar.

Tutorial

These nine activities take you from nothing to writing real Coil. Every example was compiled with `coilfe` and run on .NET; the output shown is the actual output.

1. Your first program

```
func main() -> void {
    println("Hello, Coil!");
}
```

Compile and run:

```
coilfe hello.coil -o hello.exe
dotnet hello.exe
```

Output:

```
Hello, Coil!
```

Every Coil program is a set of `func`s; an executable needs a parameterless `func main()`. `println` writes a line; `print` writes without a newline.

2. Variables and data types

```
func main() -> void {
    int    count = 42;
    double ratio = 3.5;
    bool   ok    = true;
    string name  = "Coil";
    var    auto  = count * 2;          // type inferred: int
    println(count);
    println(ratio * 2.0);
    println(ok && (count > 10));
    println(name);
    println(auto);
}
```

Output:

```
42
7
True
Coil
84
```

Each declaration becomes one IL local. `var` infers the type from the initializer (`auto` is `int`). Note the .NET formatting: the `bool` prints `True`.

3. Flow control

```
func main() → void {
    int n = 1;
    while (n ≤ 5) {
        if (n % 2 == 0) { println(n); } else { println(0 - n); }
        n = n + 1;
    }
}
```

Output:

```
-1
2
-3
4
-5
```

`while` is the only loop. `if` / `else` lower to `brfalse` to skip the `then` block and `br` to jump over the `else`; comparisons (`≤` , `==`) and `&&` / `||` produce the `bool` the branch tests.

4. Arrays

Coil has **no array type** — it is part of the minimal subset (see *Subset boundaries*). The honest workaround for sequence problems is to compute over an *index range* with `while`, never materializing storage. For example, the sum of the first `n` squares:

```
func sumSquares(int n) → int {
    int total = 0;
    int i = 1;
    while (i ≤ n) { total = total + i * i; i = i + 1; }
    return total;
}
func main() → void { println(sumSquares(5)); }   // 1+4+9+16+25
```

Output:

```
55
```

When you genuinely need indexed storage, the intended path is to compile Coil to a `.dll` and let a C#/VB.NET host own the arrays, calling Coil for the per-element math (Activity 9). At the IL level an array would be `newarr` + `ldelem` / `stelem`; adding the syntax is a natural extension but is not in the minimal language.

5. Subroutines and functions

```
func add(int a, int b) -> int { return a + b; }

func fib(int n) -> int {
    if (n < 2) { return n; }
    return fib(n - 1) + fib(n - 2);
}

func factorial(int n) -> int {
    int result = 1;
    int i = 1;
    while (i ≤ n) { result = result * i; i = i + 1; }
    return result;
}

func main() -> void {
    println(add(2, 3));
    println(fib(10));
    println(factorial(5));
}
```

Output:

```
5
55
120
```

Functions take typed parameters and an optional `-> rettype`. They may recurse (`fib`) and call each other in any order. A call is just `call` after pushing the arguments — `add(2, 3)` is `ldc.i4.2 ; ldc.i4.3 ; call add`.

6. Memory management

Coil exposes the CLR's model directly, so "memory management" *is* the IL execution model — and there is no manual allocation to manage:

- **The evaluation stack.** Every expression pushes its value; every operator pops its operands and pushes a result. `(a + b) * c` is `ldarg a ; ldarg b ; add ; ldarg c ; mul`.
- **Locals and arguments** live in fixed IL slots: a declaration is `... stloc x`, a use is `ldloc x`; parameters are `ldarg` / `starg`. No heap is involved for `int` / `double` / `bool` — they are value types on the stack.
- **Strings** are managed `System.String` references; they are allocated and garbage-collected by .NET. You never free anything.

`factorial` above compiles to two IL locals (`result`, `i`) and a `while` loop whose guard is `ldloc i ; ldarg n ; cgt ; not ; brfalse`. You can see the whole program's IR — the textual stack IL — beside any compile:

```
coilfe factorial.coil -o factorial.exe   # writes factorial.ir next to it
```

7. Strings and a text layout

Coil has string `+` (concatenation) and `while`, which is enough to build a simple right-aligned column. With no built-in string functions, you write the helpers — and that is the point of the exercise:

```
func digits(int n) → int { int d = 1; while (n ≥ 10) { n = n / 10; d = d + 1; } return d;
func spaces(int n) → string { string s = ""; int i = 0; while (i < n) { s = s + " "; i = i
func main() → void {
    int i = 1;
    while (i ≤ 4) {
        int cube = i * i * i;
        println(spaces(4 - digits(cube)) + "" + cube);    // "" + cube ⇒ int joined as strir
        i = i + 1;
    }
}
```

Output (a column of cubes, right-aligned to width 4):

```
   1
   8
  27
  64
```

`"" + cube` concatenates an `int` onto a string: the `int` is boxed and joined with `String.Concat`, so numbers compose with text naturally.

8. Drawing a picture

Coil has no graphics, so this is **text art** — a triangle drawn with nested `while` loops and `println`:

```
func main() → void {
    int row = 1;
    while (row ≤ 5) {
        string line = "";
        int k = 0;
        while (k < row) { line = line + "*"; k = k + 1; }
        println(line);
        row = row + 1;
    }
}
```

Output:

```
*
**
***
****
*****
```

9. Where to go next

The reason Coil exists is interop. Compile a library and call it from C#:

```
// lib.coil
func add(int a, int b) → int { return a + b; }
func fib(int n) → int { if (n < 2) { return n; } return fib(n - 1) + fib(n - 2); }
func circleArea(double r) → double { return 3.14159265358979 * r * r; }
func greet(string who) → string { return "Hello, " + who + "!"; }
```

```
coilfe lib.coil -o coillib.dll --dll
```

```
// C# host (references coillib.dll)
Console.WriteLine(CoilProgram.add(2, 3));        // 5
Console.WriteLine(CoilProgram.fib(15));          // 610
Console.WriteLine(CoilProgram.circleArea(3.0)); // 28.274333882308113
Console.WriteLine(CoilProgram.greet("C#"));      // Hello, C#!
```

VB.NET is identical (`CoilProgram.add(2, 3)`), since Coil emits ordinary `public static` methods. From here:

- **Read the IR.** Every compile drops a `.ir` next to the output — it is the stack IL. Compare it to your source to learn CIL.
- **Extend the language.** Arrays (`newarr` / `ldelem`), `for` loops, or calling .NET methods are each a handful of grammar rules in `coil.y` and a handful of IR ops in the `coilasm` assembler.